

SRI SRI UNIVERSITY

Machine Learning | Project Presentation

Movie Recommendation System

Content-based filtering with cosine similarity

Guided by: Ansh Agrawal

TEAM MEMBERS

1 **Shakti Subham Mishra**
1ETBTA20251040

2 **Apurba Mohanty**
1ETBTA20251039

3 **Preksha Singh**
1ETBTA20251038

4 **Puravi Dash**
1ETBTA20251041

PROBLEM & MOTIVATION

Too many movies, too little time

1

Choice overload

Thousands of titles on every platform. Users scroll, get tired and leave.

2

Search is not enough

You can only search for a movie you already know. Discovery needs suggestions.

3

Businesses need it

Good suggestions keep users watching, and every major streaming service relies on them.

PROBLEM STATEMENT

Given one movie a user liked, recommend the 5 most similar movies, automatically and with a model simple enough to explain line by line.

Presenter: Shakti

OBJECTIVES & SCOPE

What we set out to do

1. Build a clean dataset of 110 popular movies (Hollywood, Bollywood, Tollywood, anime)
2. Represent each movie as a bag of words: genres + director + cast + keywords
3. Recommend the top 5 most similar movies using cosine similarity
4. Handle typos in titles ("toy stroy" still finds Toy Story)
5. Evaluate with Precision, Recall, F1 and Hit Rate against two baselines

IN SCOPE

Content-based recommendations

Command-line demo in Python

Offline evaluation on 20 test movies

OUT OF SCOPE

User ratings and watch history

Deep learning models

Web or mobile app

Presenter: Shakti

LITERATURE REVIEW

Three ways to recommend

Popularity-based

HOW IT WORKS

Recommend what is trending or top-rated.

VERDICT

Same list for everyone. Not personal.

Collaborative filtering

HOW IT WORKS

"Users like you also liked..." using a rating matrix.

VERDICT

Needs lots of ratings. Fails for new movies (cold start).

Content-based (ours)

HOW IT WORKS

Recommend movies whose genres, people and story are similar.

VERDICT

No user data needed, works for new movies, easy to explain.

Our choice: content-based filtering. It needs only movie information, so it fits a small offline project and every suggestion can be explained.

Presenter: Apurba

DATA

Our dataset

110
movies

20
genres

64
directors

537
unique words

Column	Example: The Dark Knight
title / year	The Dark Knight, 2008
genres	Action | Crime | Drama
director	Christopher Nolan
cast	Christian Bale | Heath Ledger
keywords	batman joker gotham vigilante superhero

Presenter: Apurba

PIPELINE

How the system works

1

Load data

movies.csv

(pandas)

2

Build tags

genres + director

+ cast + keywords

3

Vectorize

CountVectorizer

537 word counts

4

Similarity

cosine similarity

110 x 110 table

5

Recommend

sort the row,

return top 5

Example tags for The Dark Knight:

```
action crime drama christophernolan christianbale heathledger batman joker gotham vigilante superhero
```

Presenter: Apurba

THE MODEL

Cosine similarity, by hand

$$\cos(A, B) = (A \cdot B) / (|A| \times |B|)$$

1 = identical tags | 0 = nothing in common

With words appearing once each:

similarity = shared words / √(words in A × words in B)

The Dark Knight: 11 words

Batman Begins: 10 words

Shared: 7 words

7 / √(11 × 10) = 7 / 10.49 = 0.67

SHARED WORDS

- action
- christophernolan
- christianbale
- batman
- gotham
- vigilante
- superhero

Only in The Dark Knight: crime, drama, heathledger, joker

Only in Batman Begins: adventure, michaelcaine, origin

Presenter: Preksha

IMPLEMENTATION

The core code fits on one slide

```
vectors = CountVectorizer().fit_transform(df["tags"])
sim = cosine_similarity(vectors)          # 110 x 110 table

def recommend(query, df, sim, k=5):
    title = find_title(query, df)          # typo-tolerant
    i = df.index[df["title"] == title][0]
    ranked = sorted(enumerate(sim[i]), key=lambda p: -p[1])
    return [(df["title"][j], s) for j, s in ranked if j != i][:k]
```

RUN IT

```
python movie_recommender.py "The Dark Knight"
```

Python 3
language

pandas
load the CSV

scikit-learn
vectorizer + cosine

difflib
typo matching

Presenter: Preksha

RESULTS YOU CAN SEE

Live demo output

YOU LIKED

The Dark Knight

Batman Begins	**67%**
The Dark Knight Rises	60%
Joker	36%
The Prestige	29%
The Godfather	20%

YOU LIKED

Toy Story

Toy Story 3	**80%**
Inside Out	40%
Coco	37%
Finding Nemo	30%
Shrek	29%

YOU LIKED

3 Idiots

PK	**48%**
Dangal	30%
Zindagi Na Milegi Dobara	29%
Lagaan	27%
Superbad	21%

Presenter: Preksha

EVALUATION

How we measured it

20 test movies with known answers

Terminator 2 → The Terminator
Toy Story → Toy Story 3
3 Idiots → PK (26 expected movies in total)

Metrics

Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 = 2PR / (P + R)
Hit Rate@5 = test movies with ≥ 1 hit / 20

Compared: Random vs Genre-only vs Proposed

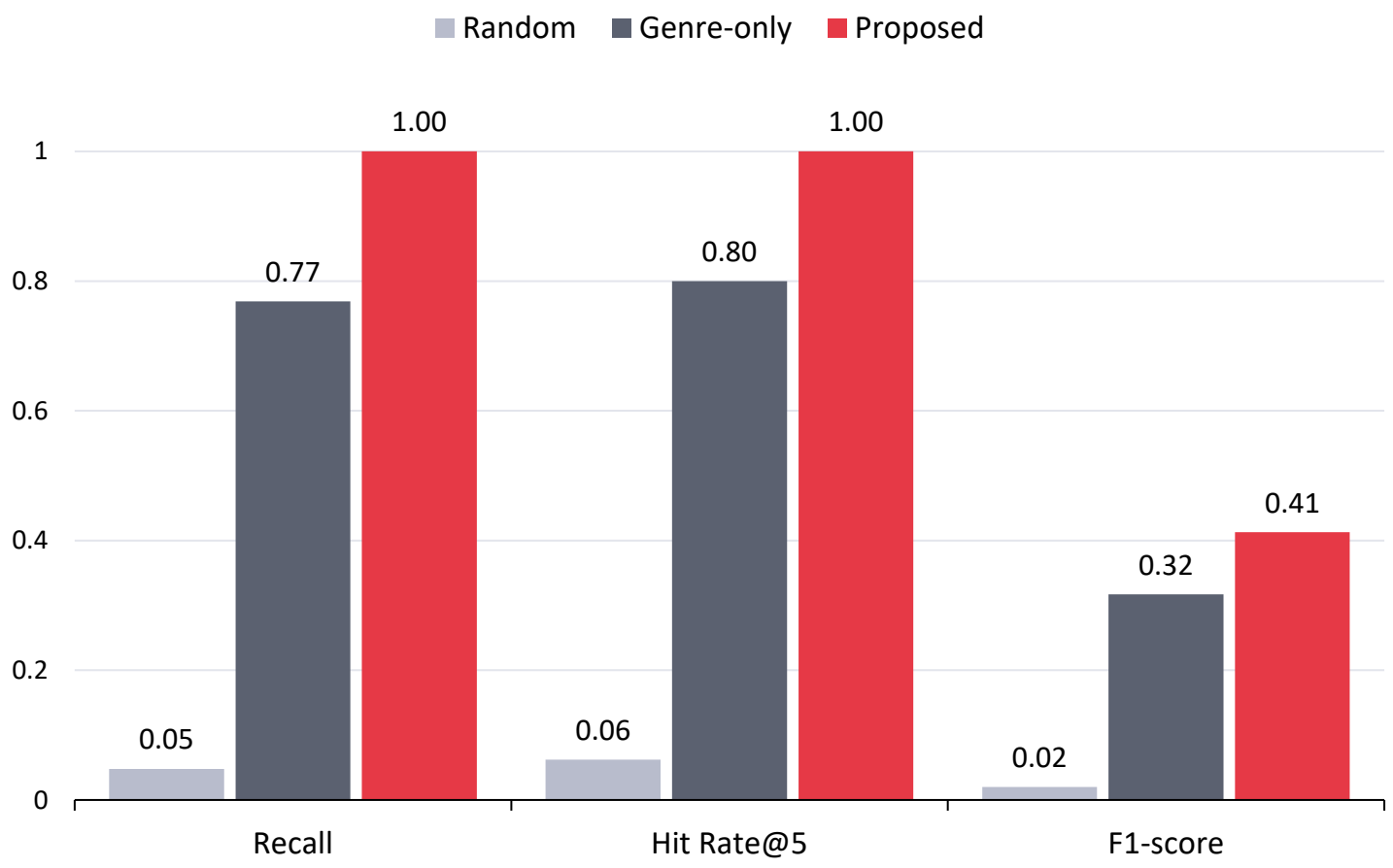
CONFUSION MATRIX (PROPOSED MODEL)

	In top 5	Not in top 5
Expected	TP = 26	FN = 0
Not expected	FP = 74	TN = 2080

Presenter: Puravi

RESULTS

Results: full tags win clearly



100%
of expected movies found (recall)

0.41
F1-score, vs 0.32 for genre-only

0.26
precision, the maximum possible here

Presenter: Puravi

DISCUSSION

Strengths, limits and what's next

Strengths

- About 100 lines, no training
- Instant results on a laptop
- No user data needed
- Works for new movies
- Every result is explainable

Limitations

- Only as good as our tags
- 'More of the same' results
- Small dataset (110 movies)
- Small, hand-made test set
- Ignores user ratings

Future scope

- Scale to MovieLens / TMDB
- TF-IDF or embeddings on plots
- Hybrid with user ratings
- Streamlit web app
- Show movie posters

Presenter: Puravi

CONCLUSION

Simple can be strong

1. A bag-of-words + cosine similarity model recommends similar movies in under a second.
2. Adding director, cast and keywords to genres raised recall from 77% to 100% on our test set.
3. The model is easy to explain, needs no user data and is a solid base for a hybrid system.

Thank you! Questions?

Shakti Subham Mishra · Apurba Mohanty · Preksha Singh · Puravi Dash | Guide: Ansh Agrawal